

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	트리아지	팀 인 원 (팀장 포함)	2명
참가부문	일반	과제유형	지정과제(SK텔레콤)

□ 결과보고서

프로젝트 개요	
프로젝트명	프롬프트 난이도 인지 기반 경량 LLM 라우터
프로젝트 등록 URL	https://github.com/leelang7/oss-sp-2026-llm-router-challenge
시연영상	https://youtu.be/ejKfuvM4vdg
프로젝트 소개	프롬프트 내용만으로 후보 모델 하나를 선택하여 주어진 예산 범위에서 답변 품질을 최대화하는 오픈소스 LLM 라우터. 모델 호출과 네트워크 접근 없이 학습된 계수와 어휘만으로 문항당 약 5밀리초에 판단.
프로젝트 세부 내용	
개발배경 및 목적	○ 추진 배경 - LLM 서비스 운영비에서 추론 비용이 차지하는 비중 최대 - 실제 트래픽의 상당수는 경량 모델로도 충분한 단순 질의 - 전량을 고성능 모델로 처리하는 관행에 따른 예산 낭비 발생 ○ 개발 목표 - 프롬프트 난이도를 자동 판별하여 최적 모델에 배정 - 동일 예산 조건에서 평균 답변 품질 향상 - 재현 가능하고 이식 가능한 오픈소스 구현 ○ 과제 제약 - 입력은 프롬프트 내용과 예산 등급으로 한정 - 모델 호출, 답변 비교, 선택 번복 불가 - 예산 초과 시 해당 등급 0점 처리
개발환경	○ 언어 및 런타임 : Python 3.11, 컨테이너 linux/arm64 (Debian slim) ○ 학습 환경 : scikit-learn 1.8.0, SciPy 1.17.1, NumPy 1.26.4 ○ 실행 의존성 : NumPy 단일 패키지 (학습용 패키지는 이미지 미포함)

	○ 공식 실행 조건 : CPU 2코어, 메모리 2GiB, 등급당 90초 ○ 격리 조건 : 네트워크 차단, 읽기 전용 루트, 비특권 사용자(UID 65532) ○ 개발 도구 : Git, Docker Buildx, 표준 라이브러리 unittest ○ 산출물 규모 : 컨테이너 이미지 73.4MB, 학습 아티팩트 5.5MB ○ 지정과제 평가 대상 커밋 : https://github.com/leelang7/oss-sp-2026-llm-router-challenge/tree/fa1268edff9a3701679ccc393b5dc34c0aabe45c - 저장소 태그 submission-2026는 결과보고서·시연영상을 포함한 제출 시점 전체를 가리킴 ○ 제출 이미지 : ghcr.io/leelang7/routerx@sha256:ea01be4aa373f1358450c56105f4f595619b7fa2bd272d418c9bc71f8b75016f
시스템 구성 및 아키텍처	○ 특징 추출 — 프롬프트 내용만 사용 - 직접 계산 특징 36종 : 길이, 한글 비율, 수식·코드 표지, 객관식 표지, 대화 구조 - TF-IDF 단어 1~2그램 60,000차원 - TF-IDF 문자 3~5그램 120,000차원 ○ 예측 — ridge 회귀 7개 헤드 - 모델별 예측 품질 3종 - 모델별 예측 출력 토큰 3종, 예측 입력 토큰 1종 - 공개 비용 정책을 적용하여 모델별 비용으로 환산 ○ 배치 선택 정책 - 예측 이득 대비 예측 비용이 큰 순서로 승격 - 동렬은 프롬프트 내용 해시로만 정렬 (문항 ID·입력 순서 미사용) - 등급별 예산 마진, 추론 모델 선택 건수 상한, 문항별 비용 상한 적용 ○ 출력 — 등급별 선택 결과 JSON (문항당 model_id 1개)
프로젝트 주요기능	□ 비용의 예측 대상 포함 - 라우팅 시점에 토큰 수가 제공되지 않아 비용 자체를 예측 대상에 포함 - 학습 단계에서도 실제 토큰 수 대신 예측값을 사용하여 추론 조건과 일치 - 로그 공간 회귀의 지수 환원 시 발생하는 평균 과소평가를 잔차 분위수로 보정 - 학습 분할의 실제 총비용을 기준으로 모델별 스케일 조정 □ 예산 초과·구조적 차단 - 추론 모델 출력 토큰 분포 : 중앙값 1,570 / 최댓값 130,504 - 단일 문항이 경량 모델 전체 비용의 26%까지 점유 가능

- 예측 비용 상한 및 분위수 회귀 방식 : 미채택
 - 사고 유발 문항이 예측 과소·실제 과대 유형이어서 필터를 통과
- 선택 건수 상한 방식 : 채택
 - 예측 오차와 무관하게 노출 규모를 제한
 - 고비용 구간은 승격 이득 대비 비용도 최저이므로 점수 손실 경미

□ 순서 비의존 배치 정책

- 예산은 배치 전체 제약이므로 문항 단위 독립 처리 불가
- 입력 순서에 의존할 경우 과제 규칙 위반에 해당
- 승격 후보를 이득·비용비로 정렬하되 동률은 프롬프트 SHA-256

해시로 판정

- 내용이 동일한 문항은 그룹 단위로 일괄 승격
- 문항 ID 및 입력 순서 변경 후 재실행 시에도 동일 결과 보장

□ 학습·실행 경로 분리

- 학습은 scikit-learn으로 수행하고 제출 이미지에는 NumPy만 포함
- TfidfVectorizer 동작을 표준 라이브러리로 재현 (소문자화, 토큰

패턴,

char_wb 패딩, sublinear tf, smooth idf, 블록별 L2 정규화)

- 두 경로의 예측 차이 $1.2e-15$ (부동소수점 한계 수준)

□ 등급별 설정 — 교차검증 기반 결정

- 2,640문항 대상 5-fold 및 8-fold 교차검증 수행
- 양쪽 fold 구성에서 예산 초과 0건인 조합만 채택
- fast : 추론모델 상한 0%, 예산 마진 0.85 (Dev 사용률 82.9%)
- balanced : 추론모델 상한 1%, 예산 마진 0.80 (Dev 사용률 80.1%)
- premium : 추론모델 상한 11%, 예산 마진 0.85 (Dev 사용률 64.7%)
- 문항별 추론모델 비용 상한 5% 병행 적용

□ 구동 방법

- 학습 : `python train_routerx/train.py --artifact`

`src/routerx/artifact.npz`

- 라우팅 : `python -m routerx.cli --input INPUT --tier TIER --output OUTPUT`

- 자체 점검 : `python -m routerx.audit --input INPUT --outcomes OUTCOMES`

- 컨테이너 : 공식 자원 한도 조건에서 실행 확인 완료

□ 자체 점검 결과 — 공개 Dev 880문항

- ID·순서 불변성 : 400문항 불일치 0건
- 결정성 : 300문항 불일치 0건
- 엡지 케이스 7종 통과 (빈 프롬프트, 초장문, 어휘 외 문자, 이모지, 장문 대화)

	- 실행 시간 : 등급당 4.2초 (한도 90초 대비 5%) - 예산 : 3개 등급 전량 통과, 여유 17~35% - 단위 시험 21건 통과 □ 성능 - 공개 Dev 880문항 : 최종 0.7167 - 교차검증 2,640문항 : 0.6643 (예산 초과 0건) - 공식 최고 baseline 0.6954 / 전량 경량 모델 0.6193
기대효과 및 활용분야	○ 이식성 - 특정 모델군에 비종속하며, 후보 모델의 품질·토큰 기록과 비용 정책만으로 임의의 모델 조합에 재학습 가능 - 학습 스크립트 및 평가 도구를 함께 공개하여 사내 모델 구성 기준 재현 지원 ○ 운영 적합성 - 네트워크 및 GPU 불필요, CPU 2코어에서 동작하여 사내망·온프레미스 즉시 적용 - 아티팩트 5.5MB로 엣지 환경 탑재 가능 ○ 기대효과 - 고객센터·검색·공공 AI 서비스 등 대규모 트래픽 환경의 추론 비용 절감 - 동일 예산에서 품질 향상 또는 동일 품질에서 비용 절감 - 라우팅 정책·학습 파이프라인·평가 도구 공개로 관련 연구의 재현 기준점 제공
기타	□ 혁신성 및 차별성 - 예산 초과를 예측 정확도가 아닌 구조적 장치로 차단 · 예측 기반 방어에 실패 원인을 실험으로 규명한 후 예측 무관 장치로 대체 - 순서 불변성을 설계 단계에서 보장 · 동률 정렬 키를 프롬프트 내용 해시로 지정하여 감사 항목을 구조적으로 충족 - 학습·실행 경로의 일치를 수치로 증명하고 시험으로 고정 (차이 $1.2e-15$) - 제출 전 자체 점검 도구를 함께 제공 · 규칙 준수, 결정성, 엣지 케이스, 실행 시간, 예산 여유를 일괄 확인 - 정보 수준별 도달 가능 상한을 분석하여 개선 여지를 정량화 · 전량 경량 0.605 / 비용 정확 0.683 / 이득 완전정보 0.725 / 완전정보 0.792 - 측정 결과에 근거한 기술 선택 및 설계 회귀 2건

· 부스팅 트리 : 단일 측정 최고점이나 교차검증 예산 미충족으로 미채택

· 문장 임베딩 : 예측 성능 저하(0.425 → 0.372)로 미채택

□ 한계점

- 경량·중형 모델 간 품질 차이는 예측 상관 0.01~0.10으로 사실상 예측 불가

· 점수가 생성 2~4회 평균값이어서 확률적 변동이 지배적

· 부스팅 트리 및 다국어 임베딩 적용으로도 한계 미극복

- 안전 확보를 위해 예산의 17~35%를 미사용 상태로 유지

- 컨테이너 이미지 73MB (NumPy 의존)

□ 향후 발전 로드맵

- 1단계 : 출력 길이 예측 정확도 개선을 통한 예산 활용률 제고

- 2단계 : 실시간 서버용 온라인 예산 컨트롤러 구현

- 3단계 : 타 모델군 및 비용 정책으로의 이식 사례 확보

- 4단계 : 생성 반복 횟수가 확대된 자료 확보 시 품질 예측 재검증

□ 소감 및 후기

본 과제는 성능 향상보다 안정성 확보에 더 많은 시간이 소요되었다. 예산을 최대한 활용하는 정책은 공개 자료에서 우수한 결과를 보였으나 데이터 분할을 달리하면 한도를 초과하였고, 그때마다 해당 등급의 점수가 전부 무효화되었다. 평균값을 조정하는 마진 설정만으로는 분포의 두꺼운 꼬리를 통제할 수 없다는 사실을 여러 차례의 실패를 거쳐 확인한 뒤에야, 위험을 예측하려 시도하는 대신 노출 규모 자체를 제한하는 방향으로 설계를 전환할 수 있었다.

학습 코드와 실행 코드가 분리된 지점에서 예측이 어긋난 결함 또한 기억에 남는다. 공식 검증 절차는 통과하였으나 실제로는 세 등급 모두 예산을 초과하는 상태였으며, 자체 점검 도구를 별도로 마련하지 않았다면 제출 이후에야 발견하였을 것이다. 검증 절차를 코드로 남기는 작업이 기능 구현 못지않게 중요하다는 점을 체감하였다.

측정 결과가 예상과 어긋날 때 이를 수용하는 판단도 쉽지 않았다. 부스팅 트리와 문장 임베딩은 모두 개선이 기대되는 접근이었으나 교차검증 과정에서 각기 다른 이유로 부적합함이 확인되었고, 단일 측정치가 아닌 반복 검증 결과에 근거하여 두 차례 설계를 되돌렸다. 본 대회를 준비하며 오픈소스 프로젝트가 갖추어야 할 재현성과 검증 체계의 의미를 다시 생각하게 되었다.

붙임1

SBOM[소프트웨어 자재명세서]

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	libgfortran	5.0.0	GPL-3.0-with-GCC-exception	https://github.com/gcc-mirror/gcc	Fortran 런타임 / NumPy 휠 번들, 동적 링크
2	NumPy	1.26.4	BSD-3-Clause	https://github.com/numpy/numpy	실행 의존성. 행렬 연산·배치 선택 / 라이브러리로 불러 씀
3	OpenBLAS	0.3.23.dev	BSD-3-Clause	https://github.com/OpenMathLib/OpenBLAS	행렬 연산 가속 / NumPy 휠 번들, 동적 링크
4	CPython	3.11.16	PSF-2.0	https://github.com/python/cpython	실행 런타임 / python:3.11-slim-bookworm
5	scikit-learn	1.8.0	BSD-3-Clause	https://github.com/scikit-learn/scikit-learn	학습 전용. TF-IDF 어휘 생성·ridge 학습 / 라이브러리로 불러 씀
6	SciPy	1.17.1	BSD-3-Clause	https://github.com/scipy/scipy	학습 전용. 희소 행렬 연산 / 라이브러리로 불러 씀
7	PyArrow	23.0.1	Apache-2.0	https://github.com/apache/arrow	자료 준비 전용. 원본 변환 / 라이브러리로 불러 씀
8	ossp_router (과제 제공)	v1	Apache-2.0	https://github.com/sktelecom/ossp-2026-llm-router-challenge	입출력 규격·채점 도구 / 라이브러리로 불러 씀

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☒ 표시)

- ☐ 유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동한 경우)
- ☐ 유형 2: 외부 모델 파인튜닝 (기존 공개 모델을 가져와 준비한 데이터셋으로 추가 미세조정된 경우)
- ☒ 유형 3: 자체 개발 모델 (기반 모델 없이 참가팀이 처음부터 가중치를 직접 전체 학습시킨 경우)
- ※ 개발 과정에서 코딩 및 디버깅 보조용으로 상용 AI(GPT, Claude 등)를 단순 활용한 경우는 체크하지 않음(4번 항목에 기재)

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음 (기반 모델 없이 자체 학습)	기반 모델 라이선스	해당 없음
--------------	------------------------	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	과제 제공 공개 Train 1,760문항 및 Dev 880문항 (총 2,640문항). 프롬프트와 모델별 품질·토큰 사용량으로 구성. 출처 및 라이선스는 과제 저장소 THIRD_PARTY_NOTICES.md에 공개.
데이터 정제/ 가공 방법 요약	과제 제공 materialize_public_data.py 절차를 그대로 사용. 개인정보가 포함되지 않은 공개 벤치마크 자료로 별도 비식별화 대상 없음. 텍스트는 소문자화 및 토큰화만 수행하고 원문 미변형.
새로 생성된 가중치 공개 저장소 URL	https://github.com/leelang7/ossp-2026-llm-router-challenge 의 src/routerx/artifact.npz (승인 절차 없이 접근 가능, 수상 시 5년간 공개 유지)
가중치 파일 정보 및 배포방식	파일명 : artifact.npz / 전체 가중치 배포 (ridge 회귀 7개 헤드, 1.44M 파라미터) / 용량 5.5MB / 저장소 직접 커밋 내용 : 회귀 계수 및 절편, TF-IDF 어휘·IDF(단어 60,000 · 문자 120,000), 특징 정규화 통계, 등급별 안전계수 및 상한값

4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)

직접 작성한 코드의 오픈소스 라이선스	Apache License 2.0	학습/추론 소스코드 공개 저장소 URL	https://github.com/leelang7/oss-2026-llm-router-challenge (공개 유지 조건 준수)
상용 AI 보조도구 활용 여부 및 범위	코드 작성, 실험 설계, 디버깅 보조에 Claude(Anthropic)를 활용. 알고리즘 설계와 실험 결과 해석, 채택 여부의 최종 판단은 참가자가 수행.		